
University of Messina



Bachelor's in data Analysis
Academic year -2024/2025
Machine Learning
Project Report

Supervisor:

Prof. Giacomo Fiumara

Students:

Pham Gia Khiem- 551026 (Vietnam)

Mohammed Hassan-541140 (Bangladesh)

Fazlur Rahman- 541927 (Bangladesh)

Table of Content

- 1.1. Dataset Exploration
- 2.1. Data Preprocessing
- 2.2. A Cleaned Dataset
- 3.1. Explanatory Data Analysis
- 4.1. Feature Engineering
- 4.2. A Dataset with new features
- 5.1. Comparison of different model performance
- 5.2. Trained Models
- 5.3. Hyperparameter tuning
- 6.2 Best tuned models

Introduction:

This project aims to explore the end-to-end process of building robust ML models, starting from raw data and culminating in actionable insights. The objective is to demonstrate a systematic approach to data analysis, cleaning, exploration, feature engineering, and model development, followed by thorough evaluation and interpretation.

Understanding the Dataset

The dataset used in this project consists of 9000 rows and 11 columns. The columns are as follows:

- **Feature_1 to Feature_8:**
These are numerical features representing the primary attributes of the dataset.
- **Category_1 and Category_2:**
These are categorical features that capture distinct classifications within the data.
- **Target:**
This is the dependent variable, which the model aims to predict.

```
df = pd.read_csv('data/final_project_dataset_complete.csv', index_col = False)
df
```

	feature_1	feature_2	feature_3	feature_4	feature_5	feature_6	feature_7	feature_8	category_1	category_2	target
0	0.496714	1.146509	-0.648521	0.833005	0.784920	-2.209437	-1.300105	-2.242241	Above Average	Region C	1
1	-0.138264	-0.061846	NaN	0.403768	0.704674	-2.498565	-1.339227	-1.942298	Below Average	Region A	0
2	0.647689	1.395115	-0.764126	1.708266	-0.250029	1.956259	1.190238	1.503559	High	Region C	1
3	1.523030	2.657560	-2.461653	2.649051	0.882201	3.445638	2.120913	3.409035	High	Region B	1
4	-0.234153	-0.499391	0.576097	-0.441656	0.610601	0.211425	0.935759	-0.401463	Below Average	Region C	0
...	...	...	...	...	...	...	...	...	...	...	...
8995	0.101630	0.400250	NaN	-0.019412	-0.063150	0.077627	0.540975	-0.169030	Above Average	Region A	0
8996	1.167218	2.177774	-1.716067	1.994835	0.350043	-0.544915	0.089050	-0.944220	High	Region C	1
8997	1.588447	3.333945	-2.615488	3.476880	-0.933276	2.027658	0.239583	1.951491	High	Region C	1
8998	-0.684987	-1.599835	1.063341	-1.252109	-0.724001	2.228943	0.989794	1.936476	Low	Region B	0
8999	0.801182	1.808647	-1.383917	1.841776	0.702840	-0.339918	-0.027882	-0.462744	High	Region C	1

9000 rows × 11 columns

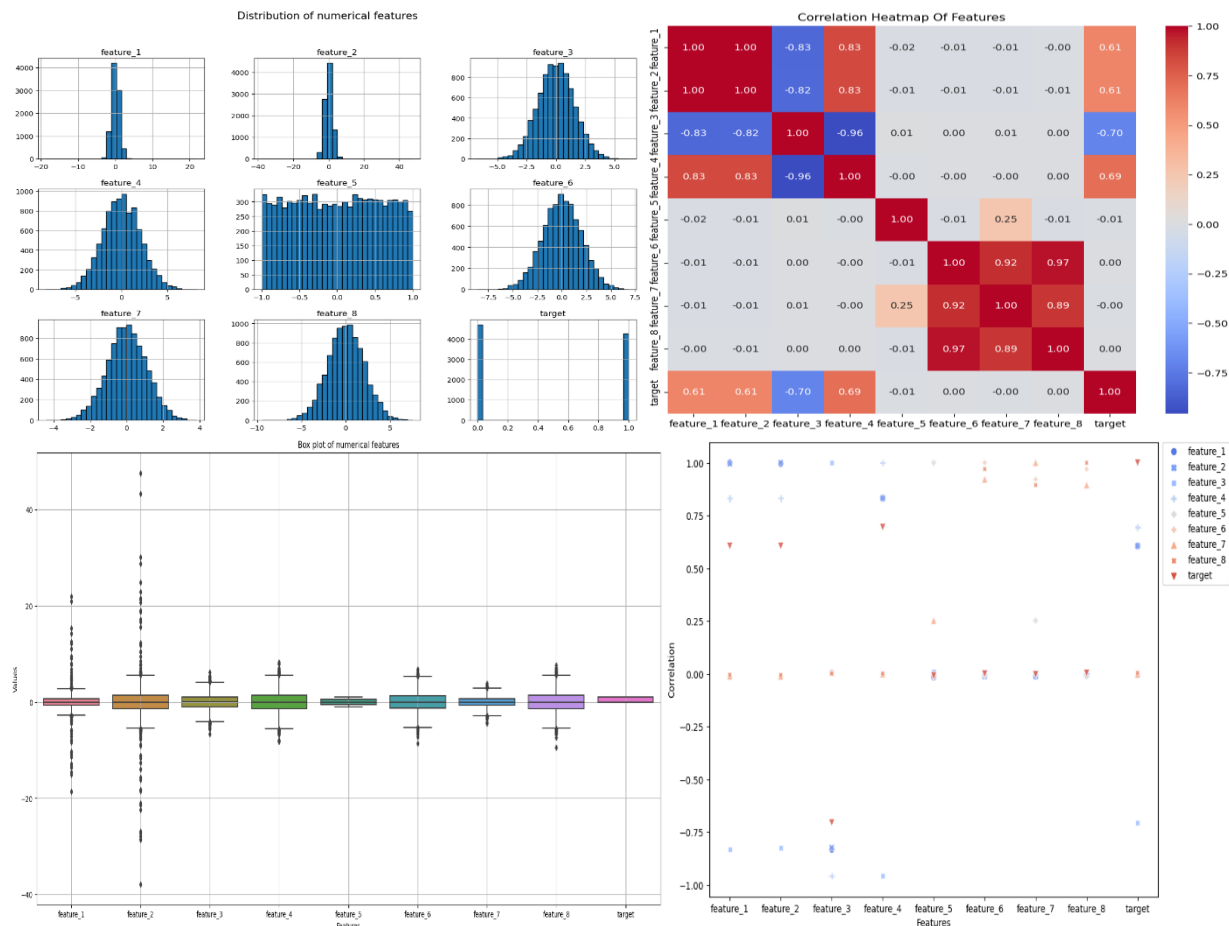
Basic Information and Statistical Details about the Dataset:

	feature_1	feature_2	feature_3	feature_4	feature_5	feature_6	feature_7	feature_8	target
count	9000.000000	9000.000000	8600.000000	9000.000000	9000.000000	8500.000000	9000.000000	9000.000000	9000.000000
mean	0.000427	0.003349	0.003235	-0.008481	-0.002177	-0.006447	0.000592	0.003348	0.475444
std	1.241318	2.508324	1.542901	2.061784	0.577415	1.981615	1.075064	2.043643	0.499424
min	-18.665400	-37.852816	-6.676680	-8.190124	-0.999791	-8.590782	-4.422265	-9.474989	0.000000
25%	-0.680062	-1.382610	-1.022085	-1.399928	-0.502614	-1.329040	-0.700078	-1.356620	0.000000
50%	-0.003938	-0.016698	0.005196	-0.019541	0.001695	-0.003137	-0.000097	-0.007584	0.000000
75%	0.680513	1.380228	1.038571	1.394151	0.497004	1.324897	0.731942	1.402024	1.000000
max	21.934496	47.603454	6.203055	8.189001	0.999914	6.803751	3.857219	7.572578	1.000000

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9000 entries, 0 to 8999
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   feature_1       9000 non-null   float64
1   feature_2       9000 non-null   float64
2   feature_3       8600 non-null   float64
3   feature_4       9000 non-null   float64
4   feature_5       9000 non-null   float64
5   feature_6       8500 non-null   float64
6   feature_7       9000 non-null   float64
7   feature_8       9000 non-null   float64
8   category_1      9000 non-null   object
9   category_2      9000 non-null   object
10  target          9000 non-null   int64
dtypes: float64(8), int64(1), object(2)
```

Here, we observed the statistical details of each column, which provided insights into the data types of the features. This understanding will assist in making necessary adjustments during the analysis.

Initial distribution of features



These visualizations collectively provide a comprehensive understanding of the dataset's structure, relationships, and potential preprocessing needs. The **histograms** indicate that while some features exhibit a symmetric distribution, others follow a uniform or skewed pattern. The **correlation matrix** highlights that certain features are strongly correlated with the target variable, whereas others show little to no relationship. Additionally, it reveals the correlation (both positive and negative) between different features. The **boxplot** provides insights into data congestion and helps identify potential outliers that may need to be addressed during preprocessing. Lastly, the **scatterplots**, in alignment with the correlation matrix, further illustrate the relationships between the dataset's features and the target variable.

Data Preprocessing

Handling missing values of Numerical Features:

```
# assigning missing values to a variable
missing_values = df.isna().sum()

# Display columns with missing values
missing_values = missing_values[missing_values > 0]
print(missing_values)
```

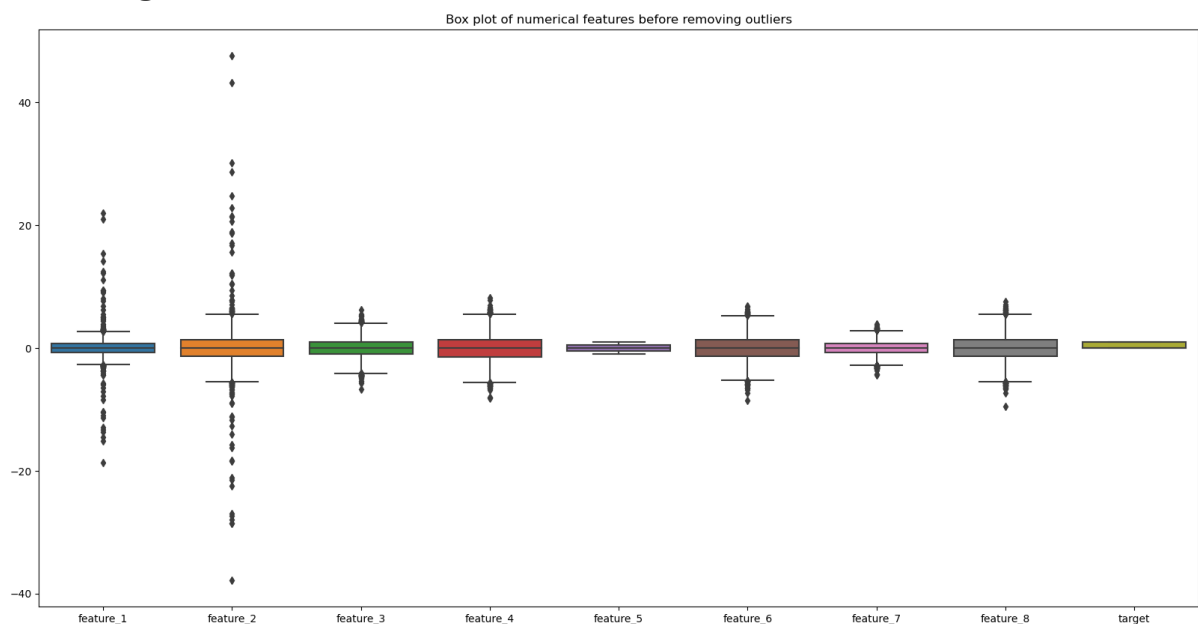
```
feature_3    400
feature_6    500
dtype: int64
```

Using (`missing_values = df.isna().sum()`), we identified approximately 400 and 500 missing values in **feature_3** and **feature_6**, respectively. To address this, we utilized the `SimpleImputer` function to replace the missing values with the median of their respective columns. Upon re-checking the dataset, we observed that there are no longer any missing values, allowing us to proceed with further analysis.

```
# Checking to see if any missing values left or not
df['feature_3'].isna().sum()
df['feature_6'].isna().sum()
```

```
Out[23]:
0
```

Handling outliers:



As shown in the **boxplot** above, there are many outliers in individual features. Therefore, we need to remove them to prevent skewing the results, ensuring improved accuracy and reliability of the model.

```
def iqr_outliers(dataset, feature_name, multiplier= 1.5):
    # Calculate IQR
    Q1 = dataset[feature_name].quantile(0.25)
    Q3 = dataset[feature_name].quantile(0.75)
    IQR = Q3 - Q1

    # Calculate Limits
    lower_limit = Q1 - multiplier * IQR
    upper_limit = Q3 + multiplier * IQR

    # Identify outliers
    outliers = dataset[(dataset[feature_name] < lower_limit) | (dataset[feature_name] > upper_limit)]

    return outliers, {'lower_limit': lower_limit, 'upper_limit': upper_limit}

outliers_detected = {}

for feature in numerical_features.columns:
    outliers, limits = iqr_outliers(numerical_features, feature)
    outliers_detected[feature] = {
        'count': len(outliers),
        'indices': outliers.index.tolist(),
        'limits': limits
    }

print(f'Feature: {feature}')
print(f'Number of outliers: {len(outliers)}')
print(f'Lower limit: {limits["lower_limit"]}, Upper limit: {limits["upper_limit"]}')
print(f'Outlier indices: {outliers.index.tolist()}')
print('-' * 50)
```

Therefore, we implement the IQR (interquartile range) method.

First, we calculate **Q1** (the 25th percentile), which represents the value below which 25% of the data points fall, and **Q3** (the 75th percentile), representing the value below which 75% of the data points fall. Next, we compute the **Interquartile Range (IQR)** using the formula:

$$\text{IQR} = \text{Q3} - \text{Q1},$$

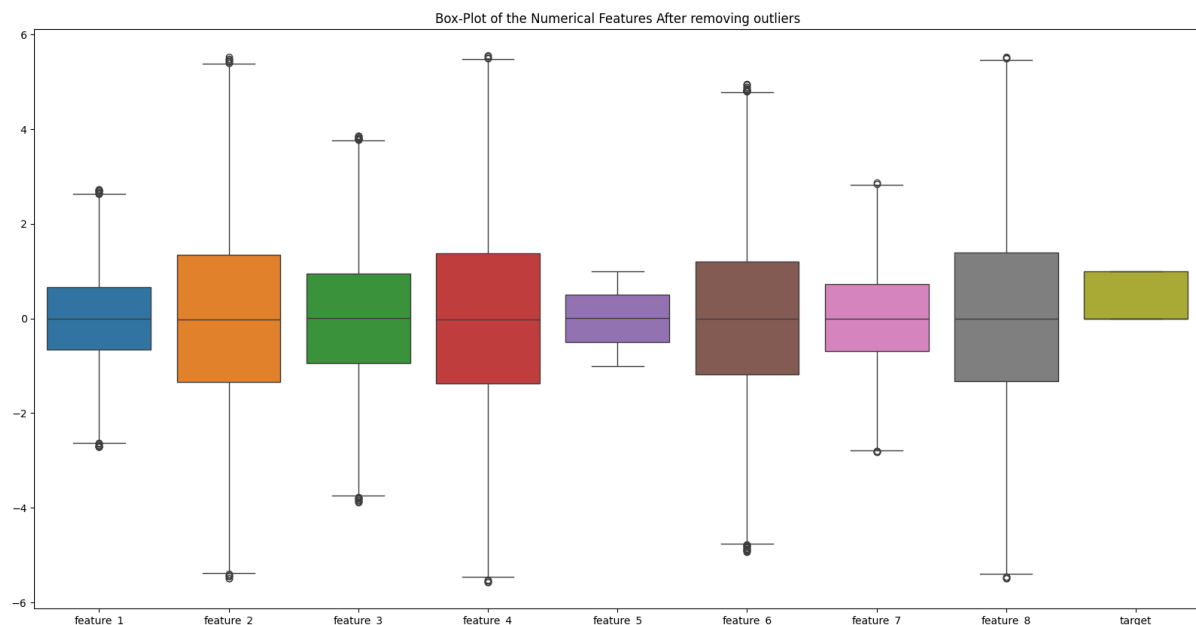
which represents the range containing the middle 50% of the data.

To determine the outlier limits, we calculate:

- **Lower Limit:** $\text{Q1} - (\text{multiplier} * \text{IQR})$
- **Upper Limit:** $\text{Q3} + (\text{multiplier} * \text{IQR})$

Any data points falling below the lower limit or above the upper limit are considered outliers.

After detecting outliers, treating them is necessary. In this case, we use a basic approach by replacing the outliers with the **median** value of their respective features.



Labelling the categorical features:

Next, we perform **label encoding** to convert categorical features into numerical form. Specifically, we apply it to **category_1** and **category_2**.

- **Category_1:**
Label Encoding is applied to this feature, where each category is assigned a unique integer. The encoded values are stored in a new column named **cat_1_encoded**.
- **Category_2:**
One-Hot Encoding is applied to this feature, creating three new columns: **Region A**, **Region B**, and **Region C**. Each row has these new columns has a value of **True** if the corresponding category is present and **False** otherwise.

```
category_columns = df_1[['category_1', 'category_2']]

for column in category_columns:
    unique_values = df_1[column].unique()
    unique_count = df_1[column].nunique()

    print(f"Column '{column}':")
    print(f"Unique values: {unique_values}")
    print(f"Number of unique options: {unique_count}")
    print("-" * 30)

Column 'category_1':
Unique values: ['Above Average' 'Below Average' 'High' 'Low']
Number of unique options: 4
-----
Column 'category_2':
Unique values: ['Region C' 'Region A' 'Region B']
Number of unique options: 3
-----
```

```
df_2 = df_1.copy()
label_encoder = LabelEncoder()
label_encoder = LabelEncoder()
df_2['cat_1_encoded'] = label_encoder.fit_transform(df_2['category_1'])
# One-Hot Encoding for category_2 (nominal)
df_one_hot = pd.get_dummies(df_2['category_2'])

# Combine the original dataframe with the one-hot encoded dataframe
df_2 = pd.concat([df_2, df_one_hot], axis=1)

# Drop the original category_2 column if no longer needed
df_2.drop(columns=['category_1', 'category_2'], inplace=True)

# Display the final dataframe
df_2.head()
```

Additionally, the column names were changed to lowercase (e.g., **Region A** to **region_a**) for consistency and to follow standard naming conventions often used in programming, which makes the dataset easier to work with in later stages.

Finally, we drop the original **category_1** and **category_2** columns, as they are no longer needed after encoding.

A Cleaned Dataset

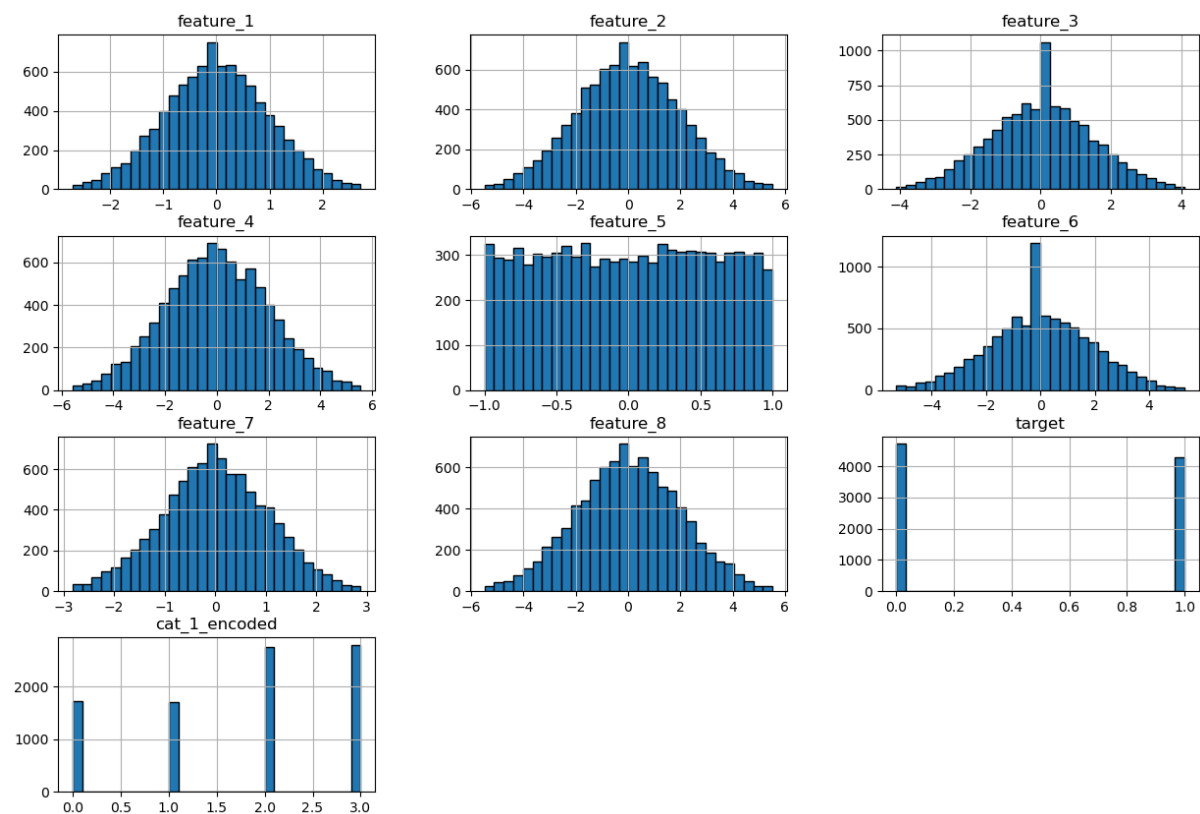
After replacing the missing values to prevent errors during model training, handling outliers to avoid skewed results, and labelling the categorical features, we obtained our cleaned dataset. This processed data was then exported as '**cleaned_dataset.csv**'.

```
df_2.to_csv('C:/Users/fazlu/OneDrive/Desktop/projects/Machine_learning_project/ML_final_project/data/cleaned_dataset.csv')
```

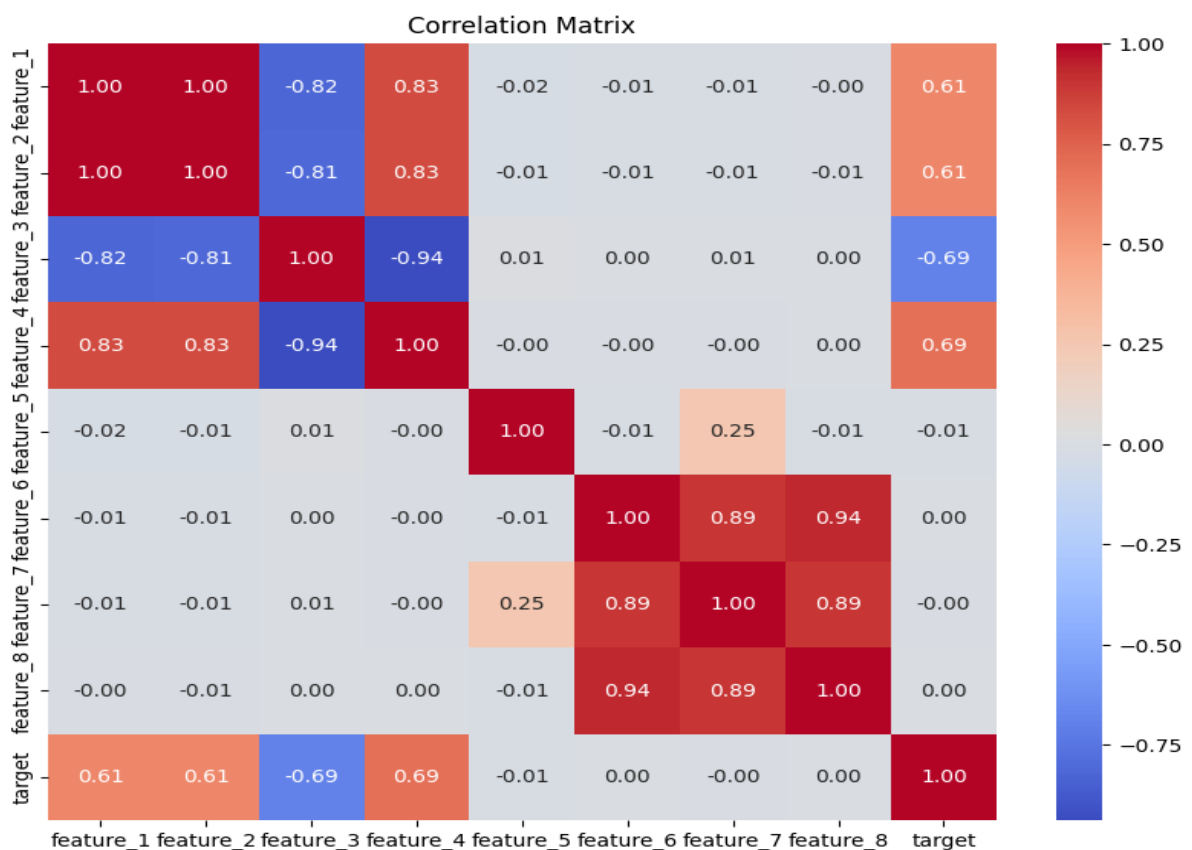
Explanatory Data Analysis

In this phase, we conducted Exploratory Data Analysis (EDA) to gain a deeper understanding of the dataset. Through this process, we analysed and visualized the data to uncover patterns, relationships, and potential issues. EDA allowed us to examine the distribution of features, identify correlations, detect outliers, and assess the overall structure of the dataset. These insights were instrumental in guiding the subsequent steps of data preprocessing, feature engineering, and model development.

Here is the distribution of features:



Implementing heatmap to visualize the correlation matrix:



Before feature engineering, the correlation matrix and histograms provided a clear understanding of which features are most important and the range within which the data is distributed. Additionally, as part of our Exploratory Data Analysis (EDA), we conducted chi-square tests for the categorical variables (**category_1** and **category_2**) and t-tests for the numerical features against the target variable.

- **Chi-Square Tests:** These tests were performed to determine if there is a statistically significant relationship between the categorical variables and the target variable. This helped us assess the relevance of these features in predicting the target.
- **T-Tests:** For numerical features, we evaluated whether the mean values differed significantly between the two target classes. This provided insight into which numerical features have distinguishing power for the target variable.

Result of T-Test and chi-square test:

	t_statistic	p_value	Category	chi2_statistic	p_value
Feature			category_1	5175.320287	0.000000
feature_1	72.253084	0.000000	category_2	1.319349	0.51702
feature_2	72.267409	0.000000			
feature_3	-90.276861	0.000000			
feature_4	91.660903	0.000000			
feature_5	-0.746804	0.455201			
feature_6	0.134524	0.892991			
feature_7	-0.189743	0.849515			
feature_8	0.441833	0.658621			
target	inf	0.000000			

Conclusion

- **Important Features:** Based on the results, **feature_1**, **feature_2**, **feature_3**, **feature_4**, and **category_1** are likely to be the most predictive features for the target variable.
- **Less Relevant Features:** Features like **feature_5**, **feature_6**, **feature_7**, **feature_8**, and **category_2** show little to no statistical relationship with the target and may be less impactful for the model.

FEATURE ENGINEERING

Raw data is often unordered, uninformative, or lacks context. Performing feature engineering carefully and insightfully can significantly improve model performance and interpretation. By creating new, relevant features and modifying existing ones, we simplify the model and reduce the risk of overfitting—where the model memorizes the training data too closely, leading to poor performance on unseen data.

```
# Performing t-tests for each numerical feature against the target variable
results = {}

for feature in numerical_features[:-1]: # Exclude the target variable
    group1 = df[df['target'] == 1][feature].dropna()
    group0 = df[df['target'] == 0][feature].dropna()
    t_stat, p_value = stats.ttest_ind(group1, group0)
    results[feature] = {'t_statistic': t_stat, 'p_value': p_value}

# Converting results to a DataFrame for better visualization
results_df = pd.DataFrame(results).T
results_df.index.name = 'Feature'

# Displaying the t-test results
print(results_df)
```

Based on our earlier observations, including insights from the correlation matrix, chi-square tests, and t-tests, we identified several features that contributed little or nothing to the target variable and were irrelevant to the model training process. Consequently, we decided to remove **feature_5**, **feature_6**, **feature_7**, **feature_8**, and **Region A**, **Region B**, and **Region C** to streamline the dataset and enhance model efficiency.

Additionally, we introduced a new feature, **feature_9**, derived from **feature_1** and **feature_4**. These two features showed a strong relationship in the correlation matrix, making it logical to combine them into a single feature. This new feature aims to better capture the combined effect of their interaction, thereby improving the model's predictive power.

Scaling the features:

After feature engineering, it's good for us to scale the features. The reason why we didn't do it before is because feature engineering often changes

scale, which means many feature engineering techniques directly change the scale of the features. That might lead to redundant and ineffective, the initial scaling step might be affected by the feature engineering step so it wouldn't be necessary to do it before hand.

```
df_4 = df_3.copy()

scaler = MinMaxScaler()

# Loop through each numerical feature and apply MinMax scaling
for feature in df_4:
    df_4[feature] = scaler.fit_transform(df_4[[feature]])

# Display the first few rows of the normalized dataset
print(df_4.head())
```

A Dataset with new features

After feature engineering the new dataset:

	feature_1	feature_2	feature_3	feature_4	target	cat_1_encoded
0	0.590734	0.602870	0.421387	0.575676	1.0	0.000000
1	0.473855	0.493050	0.501098	0.537082	0.0	0.333333
2	0.618523	0.625464	0.407291	0.654372	1.0	0.666667
3	0.779645	0.740199	0.200304	0.738960	1.0	0.666667
4	0.456204	0.453284	0.570710	0.461068	0.0	0.333333

	feature_9
0	0.580757
1	0.515752
2	0.643442
3	0.754375
4	0.458255

After performing feature engineering, we refined the original dataset by removing redundant features and introducing new ones to enhance model performance. The updated dataset now focuses on features that contribute significantly to predicting the target variable, as determined by insights from the correlation matrix, chi-square tests, and t-tests.

This new dataset is now better aligned with the goals of our analysis and serves as an optimized input for the subsequent modelling phase.

Comparison of different model performance

Model Training and Evaluation:

To evaluate model performance effectively, we first split the dataset into a training set and a testing set. The training set is used to train the models, while the testing set evaluates how well the trained models perform on unseen data, helping us avoid overfitting.

Trained Models

We experimented with the following machine learning models:

- Random Forest
- Support Vector Machine (SVM)
- Logistic Regression
- Decision Tree

Model Comparison:

Model	Training Accuracy	Testing Accuracy
Random Forest	0.9995	0.8761
SVM	0.8676	0.8683
Logistic Regression	0.8570	0.8550
Decision Tree	0.9997	0.8261

The table above compares the training and testing accuracies of the models without hyperparameter tuning. These results are based on default configurations, meaning no modifications were made to the parameters.

From the results:

- The Decision Tree model achieved the highest training accuracy (0.9997), indicating that it fits the training data very well. However, its testing accuracy (0.8261) suggests that it may overfit the training data, leading to lower performance on unseen data.
- The Random Forest model demonstrated the best testing accuracy (0.8761), indicating strong performance on unseen data. Its training accuracy (0.9995) is also comparable to that of the Decision Tree, showing it learns well from the data without overfitting.

Conclusion

Based on our observations, we selected the Random Forest model. Although both Random Forest and Decision Tree models have similar training accuracies, the Random Forest model performs significantly better on unseen data. This makes it a more reliable and effective choice for our analysis. Selecting Random Forest ensures a balanced approach to handling both training and testing data, reducing the risk of overfitting while maintaining strong predictive power.

Hyperparameter tuning

Beside default models, we also have models with hyperparameter tuning. Those are basically customized pre-built models to make it perform better as we want.

We use Grid Search for hyperparameter tuning. This technique will try all possible combinations of hyperparameter values within a specified grid. It is provided by scikit-learn libraries, so it is relatively easy to implement since it has built-in features.

Unfortunately, it still has some drawbacks like expensive computation with many hyperparameters and miss some optimal values.

Here is the comparison of the performance of the tuning models:

Model	Training accuracy	Testing accuracy
Random Forest	0.9313	0.8761
SVM	0.8693	0.8694
Logistic Regression	0.8561	0.8511
Decision Tree	0.9221	0.8583

As you can see, **Random Forest** model with hyperparameter tuning has the most training accuracy, and it also stands out to be the best one with testing accuracy. Which means that it has the best accuracy with both seen data and unseen data. This is very good and effective because it will be able to make predictions more efficient in a real-world practice. It also helps data scientists to give out more accurate and proper advice for their business.

Model Interpretation:

Random Forest: it's a very effective algorithm that is part of the ensemble learning methods. It combines the predictions of multiple decision trees to improve the overall performance of the analysis. Instead of relying on a single decision tree, this model will make a forest of many decision trees. The most frequent class predicted by the trees is chosen as the final prediction.

Default Random Forest

In [50]:

```
rf_default = RandomForestClassifier(random_state=42)
rf_default.fit(X_train, y_train)

y_train_pred = rf_default.predict(X_train)

y_test_pred = rf_default.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print(f"Training Accuracy: {train_accuracy}")
print(f"Testing Accuracy: {test_accuracy}")
```

Training Accuracy: 0.9979166666666667
Testing Accuracy: 0.87

By introducing randomness within the tree-building process, this model will reduce the risk of overfitting that can happen to each tree, it provides higher accuracy.

SVM or Support Vector Machine: is another great algorithm, its purpose is to choose the most optimal hyperplane that is best at separating data points from different classes. There are 3 types of planes which are a line if in 2D, a plane in 3D and a more complex surface within a higher dimension plane.

Default SVM

In [51]:

```
svm_default = SVC(random_state=42)
svm_default.fit(X_train, y_train)
svm_default_predictions = svm_default.predict(X_test)

y_train_pred = svm_default.predict(X_train)

y_test_pred = svm_default.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print(f"Training Accuracy: {train_accuracy}")
print(f"Testing Accuracy: {test_accuracy}")
```

Training Accuracy: 0.86875
Testing Accuracy: 0.8655555555555555

This uses the SVM with no hyperparameter tuning, means that it will use the standard built-in model of SVM to capture not the best but the most effective accuracy and from that can make predictions on the outcome.

Logistic Regression: this is a model that makes the probability of the outcome as a function of a linear combination of the input features. It provides a framework for binary classification by modelling the relationship between input features and the probability of a specific outcome. This model is very widely used thanks to its simplicity and efficiency.

Default Logistic Regression

In [57]:

```
lr_default = LogisticRegression(random_state=42)
lr_default.fit(X_train, y_train)

y_train_pred = lr_default.predict(X_train)

y_test_pred = lr_default.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print(f"Training Accuracy: {train_accuracy}")
print(f"Testing Accuracy: {test_accuracy}")
```

Training Accuracy: 0.8534722222222222
Testing Accuracy: 0.8555555555555555

Decision tree: this is another algorithm that with each decision leads to a new set of choices so that it makes predictions and data classification more easily. It begins with a single node representing a whole dataset, the algorithm selects the feature that best divides the data into subsets with similar characteristics. This is often determined using metrics like information gain or Gini impurity. Then, the root node splits into branches, each representing a possible outcome of the chosen feature. Finally, the process of splitting and creating branches continues repeatedly within each subset until a stopping criterion is done.

Default Decision Tree

In [59]:

```
dt_default = DecisionTreeClassifier(random_state=42)
dt_default.fit(X_train, y_train)

y_train_pred = dt_default.predict(X_train)

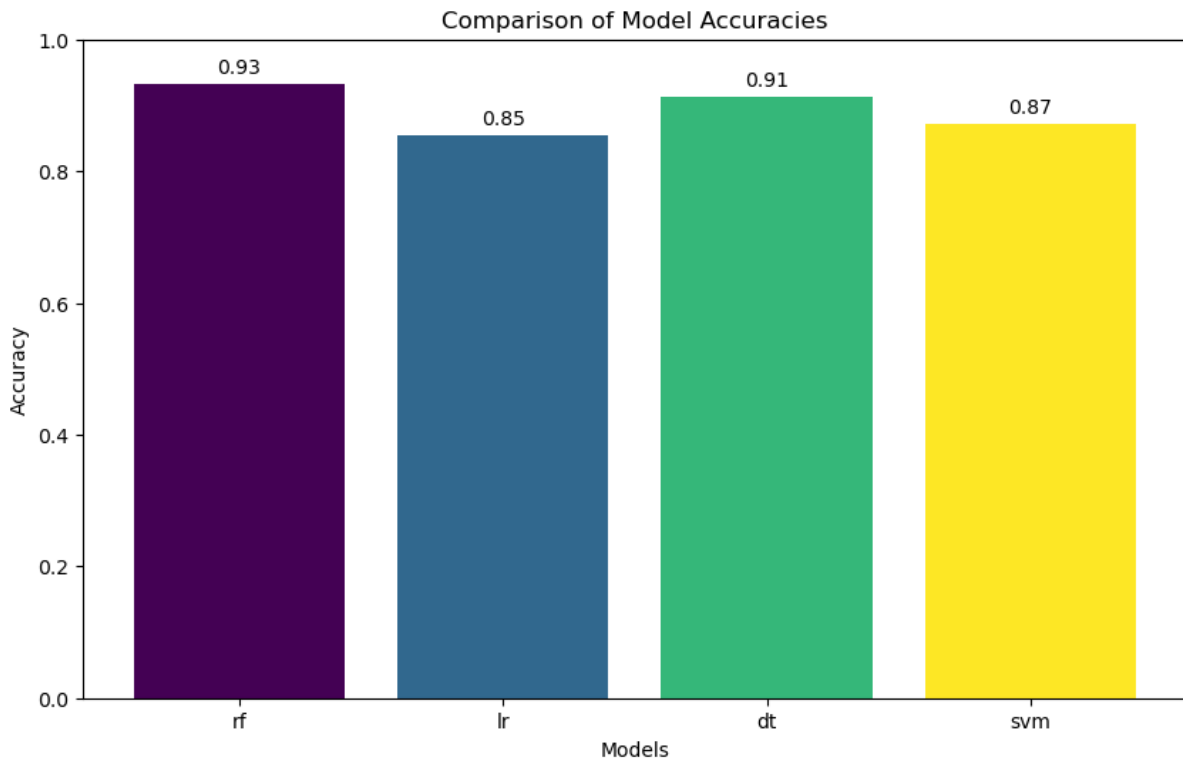
y_test_pred = dt_default.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print(f"Training Accuracy: {train_accuracy}")
print(f"Testing Accuracy: {test_accuracy}")
```

Training Accuracy: 0.9979166666666667
Testing Accuracy: 0.8294444444444444

Best Tuned Models:



The comparison above demonstrates that, after training the dataset, the Random Forest model emerges as the best performer.

Inclusion:

In this project, we have conducted a comprehensive evaluation of target variables using various machine learning models. We begin with data preprocessing, EDA, to have a deep understanding of the whole picture of the dataset. From that, we apply several techniques including feature engineering to improve models' performance and greatly enhance the output accuracy.

We have applied and evaluated various machine learning models with and without hyperparameter tuning, solving the overfitting problem.

Expansion and future applications could be used to solve problems and make precise predictions. This work is not only a final project but it's a beginning for us as students to apply this knowledge to many more complex projects in the future. We are not trying to make the "perfect" models for any problems, we are just trying to do our best to make the most efficient ones.

